

React

1.What are Hooks? (React context)

Hooks are functions in React that let you “hook into” React features (like state, lifecycle, context) inside functional components.

Before Hooks:

- State only in class components
- Lifecycle methods only in class components

After Hooks:

- Functional components can do everything classes could do — cleaner and simpler

Example:

useState

useEffect

useContext

useRef

useReducer

The core problem Hooks solve

1.Problem: Class components were messy & hard to reuse

Before Hooks, React looked like this:

```
class Counter extends React.Component {
```

```
  constructor() {
```

```
    super();
```

```
    this.state = { count: 0 };
```

```
  }
```

```
  componentDidMount() {
```

```

    console.log("Mounted");
  }

  render() {
    return (
      <button onClick={() => this.setState({ count: this.state.count + 1 })}>
        {this.state.count}
      </button>
    );
  }
}

```

Problems here:

- this confusion
- Boilerplate (constructor, bind, render)
- Lifecycle logic scattered across methods
- Logic reuse was painful

2.Problem: Logic reuse was BAD (HOCs & Render Props)

Suppose you want to reuse data fetching logic in multiple components.

Old solutions:

- HOC (Higher Order Components)
- Render Props

Both caused:

- Wrapper hell
- Hard-to-read component trees
- Props drilling issues

3.Problem: Lifecycle logic was scattered

Example:

- API call in componentDidMount
- Cleanup in componentWillUnmount
- Updates in componentDidUpdate

Same logic spread across multiple lifecycle methods

How Hooks fix these problems

1. State in functional components (useState)

```
function Counter() {  
  
  const [count, setCount] = React.useState(0);  
  
  return (  
  
    <button onClick={() => setCount(count + 1)}>  
  
      {count}  
  
    </button>  
  
  );  
}
```

No this

Less code

Easy to read

2. Side effects in ONE place (useEffect)

```
useEffect(() => {  
  
  console.log("Component mounted");  
  
  
  return () => {  
  
    console.log("Component unmounted");  
  
  };  
}, []);
```

Mount

Update

Unmount

All handled in one hook

3. Reusable logic via Custom Hooks (BIG WIN)

```
function useCounter() {  
  
  const [count, setCount] = useState(0);  
  
  const increment = () => setCount(c => c + 1);  
  
  return { count, increment };  
  
}
```

Use anywhere:

```
const { count, increment } = useCounter();
```

Clean

Reusable

No wrapper hell

4. Better separation of concerns

Instead of:

- Lifecycle-based code

You get:

- Logic-based grouping

```
useEffect(() => {
```

```
  // auth logic
```

```
}, []);
```

```
useEffect(() => {
```

```
  // analytics logic
```

```
}, []);
```

Hooks solve the problem of stateful logic reuse and complex class components by allowing state, side effects, and lifecycle behavior in functional components.